

Aula 7 - Mocking e Testes de Integração

Duração: 2 horas

Objetivos:

- Dominar o uso do Mockito para testes com dependências externas
- Implementar testes de integração com bancos reais usando TestContainers
- Gerar relatórios de cobertura de código com JaCoCo

Parte Teórica (40 min)

1. Mocking com Mockito

Quando usar?

- Testar classes que dependem de:
 - Bancos de dados
 - APIs externas
 - Serviços complexos

Principais recursos:

```
java
```

```
// Criação de mock
UsuarioRepository mockRepo = Mockito.mock(UsuarioRepository.class);

// Definindo comportamento
when(mockRepo.buscarPorId(1L)).thenReturn(new Usuario("Admin"));

// Verificando interações
verify(mockRepo, times(1)).salvar(any(Usuario.class));
```

2. Testes de Integração com TestContainers

Por que usar?

- Testar contra serviços reais (DB, Redis, Kafka)
- Eliminar mocks complexos em testes de integração

Fluxo típico:

1. Inicia container Docker com PostgreSQL
2. Executa testes
3. Destrói container

3. Cobertura de Código com JaCoCo

- Métricas importantes:
 - Linhas cobertas
 - Branches cobertos
 - Complexidade ciclomática

Parte Prática (1h20min)

Atividade 1: Mocking com Mockito

Cenário: Testar `UsuarioService` que depende de `UsuarioRepository`

1. Adicione as dependências no `pom.xml` :

xml

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>5.3.1</version>
  <scope>test</scope>
</dependency>
```

2. Implemente o teste:

java

```
class UsuarioServiceTest {

    @Mock
    UsuarioRepository repository;

    @InjectMocks
    UsuarioService service;

    @BeforeEach
    void setup() {
        MockitoAnnotations.openMocks(this);
    }
}
```

```

    }

    @Test
    void deveRetornarUsuarioQuandoExistir() {
        when(repository.buscarPorId(1L))
            .thenReturn(new Usuario("João"));

        Usuario resultado = service.buscarUsuario(1L);

        assertEquals("João", resultado.getNome());
        verify(repository, times(1)).buscarPorId(1L);
    }
}

```

Exercício:

- Teste o cenário onde `buscarPorId` retorna `null`
- Implemente um teste para o método `salvarUsuario`

Atividade 2: TestContainers com PostgreSQL

1. Adicione as dependências:

xml

```

<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>1.18.3</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <version>1.18.3</version>
    <scope>test</scope>
</dependency>

```

2. Configure o teste:

java

```

@Testcontainers
class UsuarioRepositoryIT {

    @Container

```

```

static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15");

@BeforeAll
static void setup() {
    String jdbcUrl = postgres.getJdbcUrl();
    // Configurar DataSource com a URL do container
}

@Test
void deveSalvarUsuarioNoBanco() {
    Usuario usuario = new Usuario("Maria");
    repository.salvar(usuario);

    Usuario salvado = repository.buscarPorId(usuario.getId());
    assertEquals("Maria", salvado.getNome());
}
}

```

Exercício:

- Implemente um teste para `buscarTodos`
- Adicione um teste para transações (rollback)

Atividade 3: Configurando JaCoCo

1. Adicione ao `pom.xml`:

xml

```

<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.10</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>

```

```
</executions>
</plugin>
```

2. Execute e visualize o relatório:

```
bash
```

```
mvn clean test
# Relatório em: target/site/jacoco/index.html
```

Exercício:

- Identifique uma classe com baixa cobertura e implemente testes

Desafio Integrado (30 min)

Contexto:

1. Crie um `ProdutoService` que depende de:

- `ProdutoRepository` (mockar)
- `EstoqueClient` (mockar chamada HTTP)

2. Implemente testes para:

- `reservarEstoque()` que deve:
 - Chamar `EstoqueClient.reservar()`
 - Lançar exception se estoque insuficiente

Dicas:

```
java
```

```
// Mock para HTTP Client
@Mock
EstoqueClient estoqueClient;

@Test
void deveFalharQuandoEstoqueInsuficiente() {
    when(estoqueClient.reservar(anyLong(), anyInt()))
        .thenThrow(new EstoqueInsuficienteException());

    assertThrows(EstoqueInsuficienteException.class,
```

```
() -> service.reservarEstoque(1L, 100));  
}
```

Encerramento

- **Discussão:** Quando usar mock vs container real?
- **Próxima aula:** Testes de Comportamento (BDD) com Cucumber

Tarefa:

- Implementar TestContainers para um banco Redis
- Aumentar cobertura de uma classe para >80%

Recursos:

- [Mockito Documentation](#)
- [TestContainers Guide](#)

Prontos para levar seus testes ao próximo nível? Dúvidas? É só perguntar! 🚀 💻